

EQ2330 Image and Video Processing

Project 3

Chong, Ngai Chun Adrian
ncachong@kth.se

Oğuzhan Oğuz
oguzhano@kth.se

December 31, 2025

Summary

Video data requires a large amount of bandwidth, which necessitates efficient compression algorithms. This report examines three coding architectures, from simple to complex, to address this problem. The initial Intra-frame coder isolates each frame, utilizing the Discrete Cosine Transform (DCT) to remove spatial redundancy. While effective for still images, this approach fails to exploit the similarities found between consecutive frames.

To improve performance, a Conditional Replenishment coder is implemented. This system uses a decision mechanism—governed by a Lagrangian cost function—to copy static background blocks directly from previous frames. The final architecture, Motion Compensation, advances this concept by estimating object displacement via block matching. By transmitting motion vectors rather than raw pixel data, the residual energy is significantly lowered. Results confirm that Motion Compensation yields superior rate-distortion performance. It validates the importance of temporal prediction in video processing.

1 Introduction

Digital video signals contain a vast amount of data, requiring significant bandwidth and storage capacity. Therefore, efficient video compression techniques are essential for applications such as video streaming and teleconferencing [1]. The fundamental goal of video coding is to reduce the number of bits required to represent the video signal while preserving its perceptual quality.

In this report, we consider a digital video sequence represented as $f[x, y, n]$, where x and y denote the spatial coordinates and n denotes the frame index. The problem is to encode this sequence into a bitstream and reconstruct it as $\hat{f}[x, y, n]$ with minimal distortion. We evaluate three different algorithms to achieve this: an Intra-frame video coder, a Conditional Replenishment video coder, and a Motion Compensated video coder.

First, the Intra-frame video coder encodes each video frame independently, exploiting only spatial redundancy. It divides each frame into 16×16 blocks and applies an 8×8 Discrete Cosine Transform (DCT) to each block. Afterward, the blocks are quantized and reconstructed using the inverse DCT.

Following this, a Conditional Replenishment video coder is introduced to exploit temporal redundancy. This coder selects whether to copy data from the previous frame ($n - 1$) or use intra coding for the current block at frame n . This decision is made based on minimizing a Lagrangian cost function $J = D + \lambda R$.

Finally, a Motion Compensated Video Coder (MC) is implemented. This method uses motion estimation to find a motion vector $(\Delta x, \Delta y)$ that best matches the block in the previous frame, significantly reducing the energy of the residual signal.

In this report, the rate-distortion (bitrate-PSNR) performance of all three algorithms is compared using the "Foreman" QCIF sequence, and the improvement of Motion Compensated coding over the other two coders is discussed.

2 System Description

2.1 Intra-Frame Video Coder

For this basic video coder, each 16×16 block of every frame is transformed using four 2-dimensional 8×8 DCT instances. This is achieved using Matlab functions `dct2` and `blockproc`. The coefficients x

are then quantized using step sizes Q ranging from 2^3 to 2^6 with

$$q = \text{round} \left(\frac{x}{Q} \right), \quad \hat{x} = Q q \quad (1)$$

Finally, to reconstruct each block, the Matlab function `idct2` is used to perform the inverse DCT transforms and the resultant pixel values are clamped to $[0, 255]$.

The average bit-rate per pixel is estimated using the Shannon entropy and the empirical probability mass function across all encoded blocks and frames. Each of the 256 quantized DCT coefficients in each block are considered independently and the mean is taken to produce a final combined bit-rate R_{intra} in *bits per pixel*. This could then be converted to *kbps* assuming 30 frames per second using

$$R' = \frac{R \cdot w \cdot h \cdot 30}{1000} \quad (2)$$

2.2 Conditional Replenishment Video Coder

Building upon the intra-frame video coder, a new copy mode is introduced to support conditional replenishment from the previous frame. For every block in a frame, to determine the distortion, measured using mean square error, between the new block and previous reconstructed block, the previous reconstructed frame is stored in each iteration and is initialized to all zeros.

To determine whether to use intra or copy mode, the Lagrangian cost of each mode n is calculated using

$$J_n = D_n + \lambda R_n, \quad \lambda = 0.2Q^2 \quad (3)$$

R_{intra} is found using the average bit-rate R_{intra} calculated in the previous part, adding one bit for mode selection; while R_{copy} is fixed to one bit for copy mode selection. The mode with lower Lagrangian cost is chosen for encoding the block.

The final average bit-rate per pixel is estimated by accumulating the total number of bits used for each frame with intra mode blocks using $256R_{\text{intra}} + 1$ bits and copy mode blocks using 1 bit. The total bits per frame is divided by the width and height of each frame, and the final bit-rate is the mean across all frames.

2.3 Motion Compensated Video Coder

Expanding upon the Conditional Replenishment coder, the final system incorporates Motion Compensation (MC) to exploit temporal redundancies more effectively. In this scheme, each 16×16 block is coded using one of three modes: Copy, Intra, or Inter.

The *Inter* mode utilizes block matching to find the best predictor in the previously reconstructed frame. A full-search algorithm is employed with a search range of ± 10 pixels around the current block position. The motion vectors (MV) $(\Delta x, \Delta y)$ are restricted to integer-pel accuracy. The prediction error (residual) is then encoded using the same DCT-based intra-coding method described in Section 2.1.

To select the optimal mode for each block, Rate-Distortion Optimization (RDO) is performed by minimizing the Lagrangian cost function:

$$J_{\text{mode}} = D_{\text{mode}} + \lambda R_{\text{mode}} \quad (4)$$

where $\lambda = 0.2Q^2$ remains consistent with the previous sections. The bit rates R_{mode} for each mode are estimated as follows:

- **Copy Mode:** $R_{\text{copy}} = 2$ bits (header).
- **Intra Mode:** $R_{\text{intra}} = 2$ bits (header) + $256 \cdot R_{\text{intra}}$, where R_{intra} is the estimated entropy of the intra coefficients.
- **Inter Mode:** $R_{\text{inter}} = 2$ bits (header) + 10 bits (MV) + $256 \cdot R_{\text{resid}}$. The Motion Vector is encoded using a fixed length of 10 bits (5 bits for horizontal and 5 bits for vertical components), and R_{resid} is the estimated entropy of the motion-compensated residuals. The statistics for R_{resid} were obtained by performing a pre-pass encoding of the sequence using only the Inter mode.

The mode with the minimum cost J is selected for the final bitstream.

3 Results

3.1 Distortion Measure

The quality of the reconstructed video are quantified using the Peak Signal-to-Noise Ratio (PSNR). For 8-bit pixels, the PSNR is defined as

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{d} \right) \quad [\text{dB}], \quad (5)$$

where d is the distortion measured with MSE between the original frames and the reconstructed frames.

3.2 Rate-Distortion Curves

The rate-distortion curves of all three implemented video coders can be found in Figure 1.

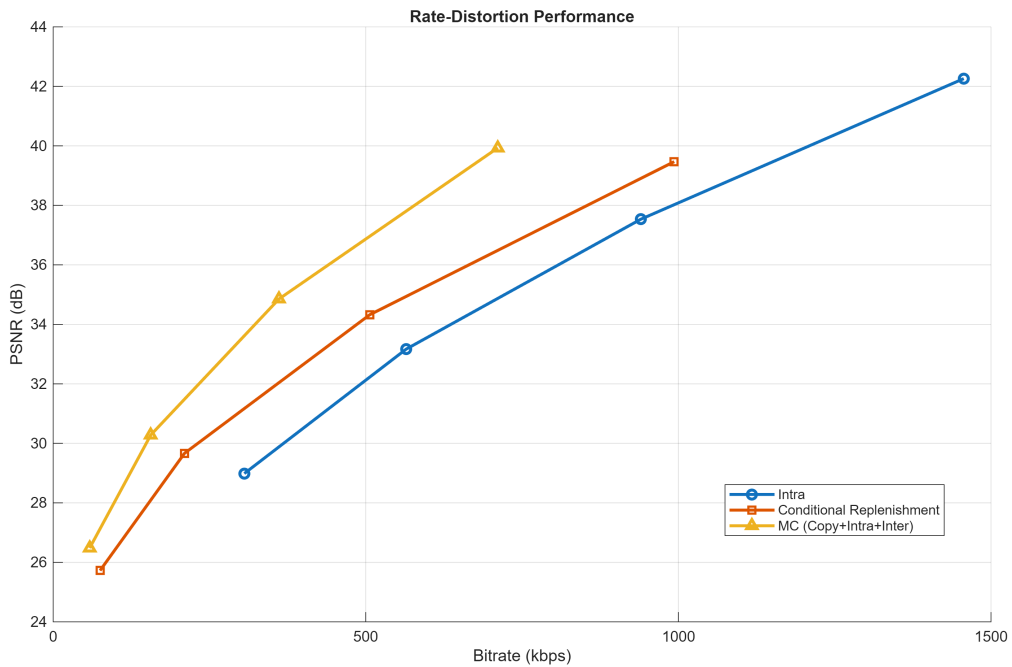


Figure 1: Rate-Distortion curves for the three implemented video coders: Intra, Conditional Replenishment, and Motion Compensation.

For the intra-frame video coder, at higher bit-rates (last two data points), the slope is found to be approximately 0.0092 dB/kbps. Converting kbps to bits per pixel, we get approximately 6.96 dB/bit, which matches the theoretical 6 dB/bit limit with some error.

As for the conditional replenishment video coder, comparing the rate-distortion curve to that of the intra-frame video coder, we can see that a lower bit-rate is needed to achieve the same video quality, i.e., equal PSNR values. However, the improvement was not significant.

In contrast, the Motion Compensated (MC) video coder significantly outperforms the Intra-Frame and Conditional Replenishment schemes. As shown in Figure 1, the MC coder achieves the highest PSNR for any given bitrate. This significant improvement comes from using motion estimation to handle changes between frames. Although encoding motion vectors adds a small overhead, it is well worth it because it drastically reduces the prediction error compared to the simpler Conditional Replenishment method.

Bonus Exercise: Mother & Daughter Sequence

In this section, we repeated the measurements using the first 50 frames of the "Mother & Daughter" QCIF sequence. This video differs significantly from "Foreman" as it contains very limited motion and a static background.

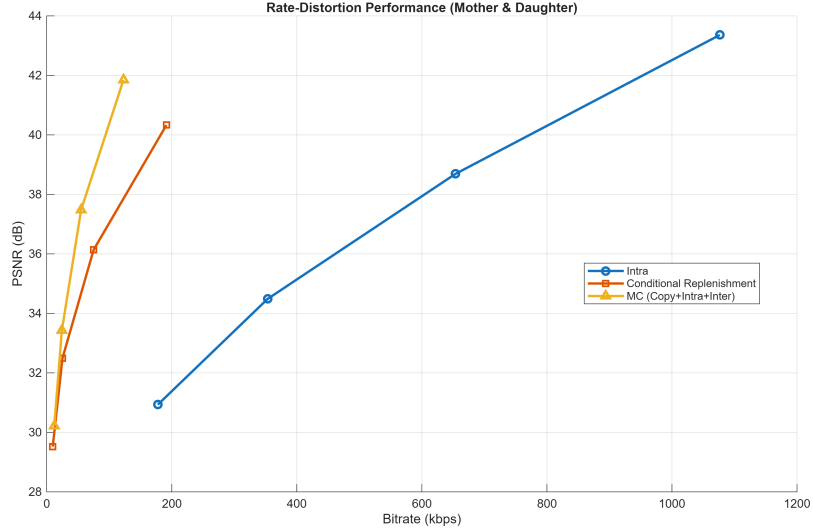


Figure 2: Rate-Distortion performance for the Mother & Daughter sequence.

The results demonstrate the important influence of video characteristics on compression performance and validate our earlier findings. The Conditional Replenishment coder outperforms Intra coding due to the sequence’s extreme staticity, which significantly lowers the bitrate.

Additionally, the Motion Compensated coder performs better than the Conditional Replenishment coder even in such a static sequence. For example, the MC coder uses a slightly lower bitrate and achieves a higher PSNR at $Q = 32$. This shows that Motion Estimation is useful for both tracking the movements of large objects and fine-tuning small displacements in scenes that are primarily static.

4 Conclusions

In this project, we tested three different video compression methods. We wanted to see how well they perform in terms of quality and bitrate.

First, the Intra-frame coder showed poor results. It treats every frame like a separate image. Because it ignores the relationship between frames, it requires a high bitrate to maintain good quality.

Next, the Conditional Replenishment method improved things slightly. It copies the static background from the previous frame. This saves some bits, but it fails to capture moving objects effectively.

Finally, the Motion Compensated (MC) coder was the clear winner. By tracking the motion of objects between frames, it achieved the highest PSNR scores. Although calculating motion vectors adds some complexity, the compression gain is massive. Our results show that handling motion is the most important part of efficient video coding.

Appendix

Who Did What

The project is divided into two parts: Chong was responsible for the implementation of the intra-frame and conditional replenishment video coders, while Oguz was responsible for the video coder with motion compensation built upon Chong's work. After completion, the results were synthesized and feedback was provided after reviewing the entire report.

Appendix: MatLab Code

1. General Helper Functions

```
% Function: Quantization
function B = quantize(A, step_size)
    B = round(A / step_size) * step_size;
end
```

```
% Function: PSNR Calculation
function psnr = psnr(d)
    psnr = 10 .* log10(255^2 ./ d);
end
```

```
% Function: Shannon Entropy Calculation
function H = shannon_entropy(X)
    X_uni = unique(X);
    P = zeros(numel(X_uni),1);
    for i = 1:numel(X_uni)
        P(i) = sum(X == X_uni(i));
    end
    P = P ./ numel(X);
    H = -sum(P .* log2(P));
end
```

2. Intra-Frame Coding Functions

```
% Function: Intra Block Encoding (DCT + Quantization)
function [coeff_quantized, block_recon, mse] = encode_block_intra(block, ...
    quantize_step_size, clip_to_8bit)

    BLOCK_SIZE = 16;
    DCT_BLOCK_SIZE = 8;

    if nargin < 3
        clip_to_8bit = true;
    end

    assert(all(size(block) == [BLOCK_SIZE, BLOCK_SIZE]), 'Input block must be 16x16');

    block = double(block);

    coeff = blockproc(block, [DCT_BLOCK_SIZE DCT_BLOCK_SIZE], @(x) dct2(x.data));
    coeff_quantized = quantize(coeff, quantize_step_size);
    block_recon = blockproc(coeff_quantized, [DCT_BLOCK_SIZE DCT_BLOCK_SIZE], ...
        @(x) idct2(x.data));
```

```

    if clip_to_8bit
        block_recon(block_recon < 0) = 0;
        block_recon(block_recon > 255) = 255;
    end

    mse = mean((block(:) - block_recon(:)).^2);
end

% Function: Intra Frame Encoding Loop
function [coeff_quantized, frame_recon, mse_values, mse] = encode_frame_intra(frame, ...
    quantize_step_size)

BLOCK_SIZE = 16;
[height, width] = size(frame);

coeff_quantized = zeros(height, width);
frame_recon = zeros(height, width);
mse_values = zeros(height / BLOCK_SIZE, width / BLOCK_SIZE);

for ii = 1:BLOCK_SIZE:height
    for jj = 1:BLOCK_SIZE:width
        block = frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);
        [coeff_quantized_block, block_recon, mse_block] = ...
            encode_block_intra(block, quantize_step_size);

        coeff_quantized(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = coeff_quantized_block;
        frame_recon(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = block_recon;
        mse_values((ii-1)/BLOCK_SIZE + 1, (jj-1)/BLOCK_SIZE + 1) = mse_block;
    end
end
mse = mean(mse_values(:));
end

% Function: Intra Bitrate Estimation
function bitrate_pel = estimate_bitrate_intra(frame_coeffs)
BLOCK_SIZE = 16;
coeffs = cat(1, frame_coeffs{:});
cw_len = zeros(BLOCK_SIZE, BLOCK_SIZE);

for x = 1:BLOCK_SIZE
    for y = 1:BLOCK_SIZE
        coeff = coeffs(x:BLOCK_SIZE:end, y:BLOCK_SIZE:end);
        cw_len(x,y) = shannon_entropy(reshape(coeff, [], 1));
    end
end
bitrate_pel = mean2(cw_len);
end

```

3. Conditional Replenishment Functions

```

% Function: Conditional Replenishment Frame Encoding
function [frame_recon, mse_values, mse, total_bits] = encode_frame_cond(frame, ...
    prev_frame, quantize_step_size, intra_bitrate_pel)

BLOCK_SIZE = 16;
lambda = 0.2 * quantize_step_size^2;

```

```

[height, width] = size(frame);

frame_recon = zeros(height, width);
mse_values = zeros(height / BLOCK_SIZE, width / BLOCK_SIZE);
total_bits = 0;

for ii = 1:BLOCK_SIZE:height
    for jj = 1:BLOCK_SIZE:width
        block = frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);
        prev_block = prev_frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);

        [~, block_recon_intra, mse_block_intra] = ...
            encode_block_intra(block, quantize_step_size);
        cost_intra = mse_block_intra + lambda * ...
            (intra_bitrate_pel * BLOCK_SIZE^2 + 1) / BLOCK_SIZE^2;

        mse_block_copy = mean((block(:) - prev_block(:)).^2);
        cost_copy = mse_block_copy + lambda / BLOCK_SIZE^2;

        if cost_intra < cost_copy
            frame_recon(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = block_recon_intra;
            mse_values((ii-1)/BLOCK_SIZE+1, (jj-1)/BLOCK_SIZE+1) = mse_block_intra;
            total_bits = total_bits + intra_bitrate_pel * BLOCK_SIZE^2 + 1;
        else
            frame_recon(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = prev_block;
            mse_values((ii-1)/BLOCK_SIZE+1, (jj-1)/BLOCK_SIZE+1) = mse_block_copy;
            total_bits = total_bits + 1;
        end
    end
end
mse = mean(mse_values(:));
end

```

4. Motion Compensation Functions

```

% Function: Motion Estimation (Block Matching)
function [pred_block, best_dx, best_dy, best_mse] = mc_predict_block(curr_block, ...
    ref_frame, ii, jj, search_range)

BLOCK_SIZE = size(curr_block, 1);
[H, W] = size(ref_frame);

dx_min = max(-search_range, 1 - jj);
dx_max = min( search_range, W - (jj + BLOCK_SIZE - 1));
dy_min = max(-search_range, 1 - ii);
dy_max = min( search_range, H - (ii + BLOCK_SIZE - 1));

best_mse = inf;
best_dx = 0; best_dy = 0;
pred_block = ref_frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);

for dy = dy_min:dy_max
    for dx = dx_min:dx_max
        cand = ref_frame(ii+dy:ii+dy+BLOCK_SIZE-1, jj+dx:jj+dx+BLOCK_SIZE-1);
        mse = mean((curr_block(:) - cand(:)).^2);
        if mse < best_mse
            best_mse = mse;
            best_dx = dx; best_dy = dy;
        end
    end
end

```

```

        pred_block = cand;
    end
end
end
end
end

```

% Function: Motion Compensated Frame Encoding (RDO)

```

function [frame_recon, mse_values, mse, total_bits, mode_map, mv_field] = ...
    encode_frame_mc(frame, prev_frame_recon, quantize_step_size, intra_bitrate_pel, ...
        inter_bitrate_pel, search_range)

```

```

BLOCK_SIZE = 16;
lambda = 0.2 * quantize_step_size^2;
mode_bits = 2; mv_bits = 10;

```

```

frame = double(frame);
prev_frame_recon = double(prev_frame_recon);
[H, W] = size(frame);

```

```

frame_recon = zeros(H, W);
mse_values = zeros(H/BLOCK_SIZE, W/BLOCK_SIZE);
mode_map = zeros(H/BLOCK_SIZE, W/BLOCK_SIZE);
mv_field = zeros(H/BLOCK_SIZE, W/BLOCK_SIZE, 2);
total_bits = 0;

```

```

for ii = 1:BLOCK_SIZE:H
    for jj = 1:BLOCK_SIZE:W
        block = frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);
        prev_block = prev_frame_recon(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);

```

```

        % Copy Mode
        D_copy = mean((block(:) - prev_block(:)).^2);
        J_copy = D_copy + lambda * (mode_bits / BLOCK_SIZE^2);

```

```

        % Intra Mode
        [~, recon_intra, ~] = encode_block_intra(block, quantize_step_size, true);
        D_intra = mean((block(:) - recon_intra(:)).^2);
        J_intra = D_intra + lambda * ((mode_bits + intra_bitrate_pel*256)/BLOCK_SIZE^2);

```

```

        % Inter Mode
        [pred, dx, dy] = mc_predict_block(block, prev_frame_recon, ii, jj, search_range);
        residual = block - pred;
        [~, residual_recon, ~] = encode_block_intra(residual, quantize_step_size, false);
        recon_inter = min(max(pred + residual_recon, 0), 255);

```

```

        D_inter = mean((block(:) - recon_inter(:)).^2);
        J_inter = D_inter + lambda * ((mode_bits + mv_bits + inter_bitrate_pel*256)/BLOCK_SIZE^2);

```

```

        [~, idx] = min([J_copy, J_intra, J_inter]);

```

```

        bi = (ii-1)/BLOCK_SIZE+1; bj = (jj-1)/BLOCK_SIZE+1;
        if idx == 1

```

```

            frame_recon(ii:ii+15, jj:jj+15) = prev_block;
            mse_values(bi, bj) = D_copy;
            mode_map(bi, bj) = 0;
            total_bits = total_bits + mode_bits;

```

```

        elseif idx == 2

```



```

        frame_recon(ii:ii+15, jj:jj+15) = recon_intra;
        mse_values(bi, bj) = D_intra;
        mode_map(bi, bj) = 1;
        total_bits = total_bits + mode_bits + intra_bitrate_pel*256;
    else
        frame_recon(ii:ii+15, jj:jj+15) = recon_inter;
        mse_values(bi, bj) = D_inter;
        mode_map(bi, bj) = 2;
        mv_field(bi, bj, :) = [dx, dy];
        total_bits = total_bits + mode_bits + mv_bits + inter_bitrate_pel*256;
    end
end
end
mse = mean(mse_values(:));
end

```

5. Inter-Mode Pre-pass Function

```

% Function: Inter-only encoding for statistics collection (Pre-pass)
function [coeff_quantized_frame, frame_recon, mse_values, mse] = encode_frame_inter_only( ...
    frame, prev_frame_recon, quantize_step_size, search_range)

BLOCK_SIZE = 16;
DCT_BLOCK_SIZE = 8;
frame = double(frame);
prev_frame_recon = double(prev_frame_recon);
[H, W] = size(frame);

coeff_quantized_frame = zeros(H, W);
frame_recon = zeros(H, W);
mse_values = zeros(H / BLOCK_SIZE, W / BLOCK_SIZE);

for ii = 1:BLOCK_SIZE:H
    for jj = 1:BLOCK_SIZE:W
        block = frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1);
        [pred, ~, ~] = mc_predict_block(block, prev_frame_recon, ii, jj, search_range);
        residual = block - pred;

        % Encode residual (no clipping here)
        [coeff_q, residual_recon, ~] = encode_block_intra(residual, quantize_step_size, false);
        recon_block = pred + residual_recon;

        % Clip at final reconstruction
        recon_block(recon_block < 0) = 0;
        recon_block(recon_block > 255) = 255;

        frame_recon(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = recon_block;
        coeff_quantized_frame(ii:ii+BLOCK_SIZE-1, jj:jj+BLOCK_SIZE-1) = coeff_q;

        bi = (ii-1)/BLOCK_SIZE + 1; bj = (jj-1)/BLOCK_SIZE + 1;
        mse_values(bi, bj) = mean((block(:) - recon_block(:)).^2);
    end
end
mse = mean(mse_values(:));
end

```

6. Main Execution Script

```

% Main Execution Script (main2.m)

```

```

clear; close all; clc;
% Test parameters
width = 176; height = 144;
frame_num = 50; frame_rate = 30;
step_sizes = 2.^(3:6);
search_range = 10;

% Import YUV
yuv_path = '../foreman_qcif/foreman_qcif.yuv';
og_frames = yuv_import_y(yuv_path, [width height], frame_num);

% Data structures
intra_bitrate_pel_all = zeros(1, length(step_sizes));
intra_bitrate_kbps_all = zeros(1, length(step_sizes));
intra_psnr_all = zeros(1, length(step_sizes));
cond_bitrate_kbps_all = zeros(1, length(step_sizes));
cond_psnr_all = zeros(1, length(step_sizes));
mc_bitrate_kbps_all = zeros(1, length(step_sizes));
mc_psnr_all = zeros(1, length(step_sizes));
inter_bitrate_pel_all = zeros(1, length(step_sizes));

% 1. INTRA Coding
for s = 1:length(step_sizes)
    Q = step_sizes(s);
    frame_coeffs_intra = cell(1, frame_num);
    mse_all = zeros(frame_num, 1);
    for f = 1:frame_num
        frame = double(og_frames{f});
        [coeff_q, ~, ~, mse] = encode_frame_intra(frame, Q);
        frame_coeffs_intra{f} = coeff_q;
        mse_all(f) = mse;
    end
    intra_bitrate_pel_all(s) = estimate_bitrate_intra(frame_coeffs_intra);
    intra_bitrate_kbps_all(s) = intra_bitrate_pel_all(s) * width * height * frame_rate / 1000;
    intra_psnr_all(s) = 10 * log10(255^2 / mean(mse_all));
    fprintf('Q=%d | Intra PSNR=%.2f dB\n', Q, intra_psnr_all(s));
end

% 2. INTER Pre-pass (Residual Estimation)
for s = 1:length(step_sizes)
    Q = step_sizes(s);
    frame_coeffs_resid = cell(1, frame_num-1);
    [~, prev_recon, ~, ~] = encode_frame_intra(double(og_frames{1}), Q);
    prev_recon = min(max(prev_recon, 0), 255);

    for f = 2:frame_num
        frame = double(og_frames{f});
        [coeff_resid, frame_recon_inter, ~, ~] = ...
            encode_frame_inter_only(frame, prev_recon, Q, search_range);
        frame_coeffs_resid{f-1} = coeff_resid;
        prev_recon = frame_recon_inter;
    end
    inter_bitrate_pel_all(s) = estimate_bitrate_intra(frame_coeffs_resid);
end

% 3. MC and Conditional Replenishment
for s = 1:length(step_sizes)
    Q = step_sizes(s);

```

```

mse_all = zeros(frame_num, 1);
total_bits = 0;

% First frame Intra
[~, prev_recon, ~, mse1] = encode_frame_intra(double(og_frames{1}), Q);
prev_recon = min(max(prev_recon, 0), 255);
mse_all(1) = mse1;
total_bits = total_bits + intra_bitrate_pel_all(s) * width * height;

for f = 2:frame_num
    frame = double(og_frames{f});
    [frame_recon, ~, mse, bits, ~, ~] = encode_frame_mc( ...
        frame, prev_recon, Q, intra_bitrate_pel_all(s), inter_bitrate_pel_all(s), search_range);
    mse_all(f) = mse;
    prev_recon = frame_recon;
    total_bits = total_bits + bits;
end

mc_bitrate_kbps_all(s) = total_bits * frame_rate / 1000 / frame_num;
mc_psnr_all(s) = 10 * log10(255^2 / mean(mse_all));
fprintf('Q=%d | MC PSNR=%.2f dB\n', Q, mc_psnr_all(s));
end

```

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Prentice Hall, Upper Saddle River, New Jersey, second edition, 2002.